COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/28/2026	**DATE SUBMITTED:**	07/28/2026
NAME:	Pangilinan Jr., Bienvenido B.	**INSTRUCTOR:**	Engr. Jimlord Quehado

ACTIVITY NO. 4 **[ACTIVITY NAME] Progress Report**

1. PROCEDURE OUTPUT

What procedures did you perform?

Source Code

```cpp
int main() {
    //create an object for the stack
    std::stack<int> newStack;

    std::cout <<  "stack empty?" << newStack.empty() << std::endl;

    // Push three elements to match your three pop sequences
    newStack.push( x: 3);
    newStack.push( x: 8);
    newStack.push( x: 15); // Added this to prevent the empty stack crash

    std::cout <<  "stack empty?" << newStack.empty() << std::endl;
    std::cout << "The size of the stack: " << newStack.size() << std::endl;

    std::cout << "The top of the element size is: " << newStack.top() << std::endl;
    std::cout << "removing element from the stack: \n";
    newStack.pop();

    std::cout << "The top of the element size is: " << newStack.top() << std::endl;
    newStack.pop();

    std::cout << "The top of the element size is: " << newStack.top() << std::endl;
    newStack.pop();

    std::cout <<  "Stack empty?" << newStack.empty() << std::endl;

    return 0;
}
```

Output

```
stack empty?1
stack empty?0
The size of the stack: 3
The top of the element size is: 15
removing element from the stack:
The top of the element size is: 8
The top of the element size is: 3
Stack empty?1

Process finished with exit code 0
```

Source Code

```
#include <iostream>

const size_t maxCap = 100;
int stack [maxCap];
int Top = -1, i, newData = 0;

void push();
void pop();
void top();
bool isEmpty();
bool isFull();


int main() {
    int choice;

    std::cout << "Enter the number of the max element for the stack: ";
    std::cin >> i;
    while (true) {
        std::cout << "stack operation" << std::endl;
        std:: cout << "1.PUSH 2.POP 3.TOP 4.isEMPTY" << std::endl;
        std:: cin >> (choice);
        switch (choice) {
            case 1: push();
                break;
            case 2: pop();
                break;
            case 3: top();
                break;
            case 4: isEmpty();
                break;
            default: std:: cout << "Wrong choice \n";
        }
```

```
        }
    }
    return 0;
}

bool isEmpty() {
    if (Top == -1) return true;
    return false;
}

bool isFull() {
    if (Top == maxCap) return true;
    return false;
}

void push() {
    if (Top == i-1) {
        std::cout << "stack overflow" << std::endl;
        return;
    }
    std::cout << "New value to add into the stack" << std::endl;
    std::cin >> newData;
    stack[++Top] = newData;
}

void top() {
    if (isEmpty()) {
        std::cout << "stack underflow" << std::endl;
        return;
    }
```

```
    }

    std::cout << "The element on the top of the stack is " << stack[Top] << std::endl;
}

void pop() {
    if (isEmpty()) {
        std::cout << "stack underflow" << std::endl;
        return;
    }
    std::cout << "popping " << stack[Top] << std::endl;
    Top--;
}
```

Output

```
Enter the number of the max element for the stack: 5

stack operation
1.PUSH 2.POP 3.TOP 4.isEMPTY
1

New value to add into the stack
10

stack operation
1.PUSH 2.POP 3.TOP 4.isEMPTY
1

New value to add into the stack
5

stack operation
1.PUSH 2.POP 3.TOP 4.isEMPTY
1

New value to add into the stack
4

stack operation
1.PUSH 2.POP 3.TOP 4.isEMPTY
1

New value to add into the stack
3
```

```
stack operation
1.PUSH 2.POP 3.TOP 4.isEMPTY
1

New value to add into the sta
2

stack operation
1.PUSH 2.POP 3.TOP 4.isEMPTY
1

stack overflow
stack operation
1.PUSH 2.POP 3.TOP 4.isEMPTY
```

Source Code (Modified)

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack [maxCap];
int Top = -1, i, newData = 0;

void push();
void pop();
void top();
bool isEmpty();
bool isFull();
void Print();


int main() {
   int choice;

   std::cout << "Enter the number of the max element for the stack: ";
   std::cin >> i;
   while (true) {
```

```
        std::cout << "stack operation" << std::endl;
        std:: cout << " 1.PUSH\n 2.POP\n 3.TOP\n 4.isEMPTY\n 5.Print\n
6.END\n " << std::endl;
        std:: cin >> (choice);
        switch (choice) {
            case 1: push();
                break;
            case 2: pop();
                break;
            case 3: top();
                break;
            case 4: isEmpty();
                break;
            case 5: Print();
                break;
            case 6:
                return 0;
            default: std:: cout << "Wrong choice \n";
        }
    }
    return 0;
}

bool isEmpty() {
    if (Top == -1) return true;
    return false;
}

bool isFull() {
    if (Top == maxCap) return true;
    return false;
}

void push() {
    if (Top == i-1) {
        std::cout << "stack overflow" << std::endl;
        return;
    }
    std::cout << "New value to add into the stack" << std::endl;
    std::cin >> newData;
    stack[++Top] = newData;
}

void top() {
    if (isEmpty()) {
        std::cout << "stack underflow" << std::endl;
        return;
    }

    std::cout << "The element on the top of the stack is " << stack[Top] <<
std::endl;
}
```

```
void pop() {
    if (isEmpty()) {
        std::cout << "stack underflow" << std::endl;
        return;
    }
    std::cout << "popping " << stack[Top] << std::endl;
    Top--;
}

// Definition added
void Print() {
    if (isEmpty()) {
        return;
    }
    for (int j = 0; j <= Top; j++) {
        std::cout << stack[j] << " ";
    }
    std::cout << std::endl;
}
```

Output (Modified)

```
New value to add into the stack
1

stack operation
 1.PUSH
 2.POP
 3.TOP
 4.isEMPTY
 5.Print
 6.END

5

5 4 3 2 1
stack operation
 1.PUSH
 2.POP
 3.TOP
 4.isEMPTY
 5.Print
 6.END

6


Process finished with exit code 0
```

Analysis
Based on my analysis, this program uses a simple array to build a stack that follows the LIFO rule. When the program starts, it asks how many numbers the stack can hold. A menu pops up on the screen so you can pick what operation to do next. When you add a number using **push()**, the program makes sure the stack is not full, adds 1 to Top, and saves your number at that position and when you remove a number using **pop()**, it checks if the stack has items, displays the top number, and it subtracts the 1 from Top to take it out. Choosing **top()** lets you view the current top number without removing it, while **isEmpty()** and **isFull()** check if the stack is completely empty or totally full. Finally, **Print()** lets you print all the variables from the stack you inputted from.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.